

0.16.3. La fonction effacer() — effacer des cases

`#include <effacer>` : `effacer(colonne, ligne, quoi)` vide des cases ; `effacer()` sans rien vide tout l'écran.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `effacer(colonne, ligne, quoi)` remet des cases à **vide**. « quoi » dit combien : un **nombre de cases**, ou le **texte lui-même** — `effacer(4, 8, "BON")` efface trois cases, le compilateur compte les lettres. Sans argument, `effacer()` vide **tout** l'écran.

La ligne à écrire : `#include <effacer>`.

Ses arguments : la colonne, la ligne, puis le nombre de cases ou le texte.

Ce qu'elle coûte : environ **120 octets** la première fois.

Essaie : remplace `3` par `"BONJ"` : quatre cases partent.

On retrouve effacer dans 95 leçons

[0.17](#), [0.17.1](#), [0.17.2](#), [0.33](#), [0.33.1](#), [0.33.2](#), [0.65](#), [0.65.1](#), [0.65.2](#), [0.66](#), [0.66.1](#), [0.66.2](#), [0.67](#), [0.67.1](#), [0.67.2](#), [0.68](#), [0.68.1](#), [0.68.2](#), [0.69](#), [0.69.1](#), [0.69.2](#), [0.76.5](#), [0.76.6](#), [0.78](#), [0.78.1](#), [0.82](#), [0.82.1](#), [0.83.2](#), [0.84](#), [0.90.1](#), [0.90.2](#), [0.90.4](#), [0.90.5](#), [0.90.6](#), [0.90.7](#), [0.90.8](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93.1](#), [0.93.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.95.1](#), [0.96](#), [0.96.1](#), [0.97](#), [0.97.1](#), [0.98](#), [0.98.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#), [1.2](#), [1.4](#), [1.6](#), [1.9](#), [1.11](#), [1.12](#), [2.3](#), [8](#), [16](#), [126](#), [134](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **345 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	26 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	96 o	← CopierTuiles — 6 lettres sur 43
"BONJOUR"	7 o	← texte() — un texte gravé

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()   × 1   11 o sur place, dans main() + EcrireTexte (10 o)
effacer() × 1    8 o sur place, dans main() + EffacerCases (12 o)
image()   × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <effacer> : effacer des cases ----
// On écrit BONJOUR, puis on efface ses trois premières cases : il reste JOUR.

#include <texte>      // pour écrire d'abord le mot
#include <effacer>    // effacer() : remettre des cases à vide
```

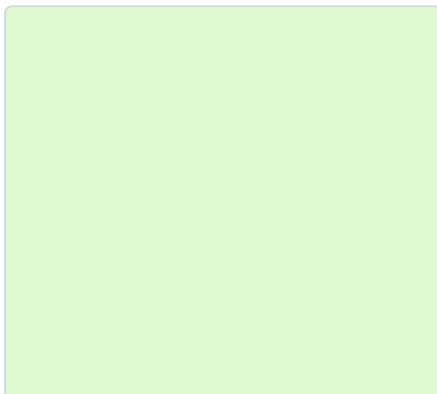
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 8, "BONJOUR");
    effacer(4, 8, 3);      // trois cases à partir de (4, 8) : B, O et N

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois cases vides, puis JOUR.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les trois premières cases sont vides

✓ JOUR reste (« JOUR »)

